Case Study: Driver Drowsiness Detection System

Problem Statement: In commercial transport and long-haul driving, driver fatigue is one of the leading causes of road accidents. Manual monitoring of driver alertness is impractical at scale, and traditional rule-based alerts often fail to catch the early signs of drowsiness.

This project aims to build an AI-powered Driver Drowsiness Detection system using Computer Vision and Deep Learning. By leveraging Transfer Learning with MobileNetV2, along with face detection tools like OpenCV and Dlib, the goal is to classify a driver's facial image as Drowsy or Non-Drowsy in real-time. The final model will be converted to TensorFlow Lite for deployment on edge devices such as in-vehicle cameras, enabling low-latency, on-device inference to enhance road safety and potentially save lives.

Dataset Link: Driver Drowsiness Data

Dataset Description: Driver Drowsiness data description

Approach Document: Approach Document for Driver Drowsiness case study

Business Questions

1. **Why not Start from Scratch:** We only have a small dataset of face images. Why is training a deep learning model from scratch a bad idea here, and what approach should we use instead?

2. **The cost of missing a Drowsy Driver:** The system can make two types of mistakes — a false alarm on an alert driver, or failing to catch a drowsy one. Which mistake is more dangerous, and which metric should we care about more?

3. **Seeing in the Dark:** A driver's cabin looks very different at noon versus midnight. How can we make sure our model doesn't struggle when lighting, shadows, and camera conditions keep changing?

4. **Why detect the Face First:** Instead of feeding the full raw image to the model, we first detect and crop the face. Why is this step important, and what could go wrong without it?

5. **Shrinking the Model for the Road:** Our trained model needs to run on a small device like an in-vehicle camera or a Raspberry Pi. How do we make the model small and fast enough for this, and what do we sacrifice in the process?

6. **Training in Two Phases:** The model is trained in two separate phases instead of all at once. Why is this two-step approach better than simply training everything from the beginning?

7. **Beyond just Accuracy:** If the model scores 95% accuracy, can we call it job done? Why or why not, especially for a safety-critical system like this?

8. **Improving the model:** After looking at your results, if the model is still missing some drowsy drivers, what are 2 practical steps you would take to fix this?

Evaluation Criteria

- **Business Story & Context:** Clarity and completeness of the problem narrative, specifically explaining why drowsiness detection matters for road safety and how AI can address this challenge in commercial transport.

- **Data Preparation & Augmentation:** Correctness of image preprocessing steps (resizing, normalization, color conversion) and the strategic use of data augmentation to ensure the model is robust to real-world variations in lighting and camera conditions.

- **Face detection and Feature extraction:** Appropriate use of OpenCV and Dlib for face detection, and clear reasoning for why isolating the face region improves model performance.

- **Transfer learning and Model architecture:** Correct implementation of MobileNetV2-based Transfer Learning, including the logical choice of the base model, the custom classification head, and the two-phase training strategy (frozen base followed by fine-tuning).

- **Model evaluation & Error Analysis:** Use of appropriate evaluation metrics (Accuracy, Precision, Recall, F1-Score) and visual tools like confusion matrices and training curves to analyze model performance and identify areas of improvement.

- **Edge deployment and TFlite Conversion:** Successful conversion of the trained model to TensorFlow Lite format, understanding of quantization, and demonstration of inference on test images using the lightweight model.

- **Notebook Clarity & Reproducibility:** Logical flow of sections, clear markdown explanations accompanying each code block, clean code structure, and ease of running the notebook end-to-end.

Submission Criteria

- **File Format:** Only PDF files are allowed for submission.
- **File Size Limit:** The PDF must be less than 20MB in size.
- **Solution Notebook:** All the business questions are also to be answered in the same Colab notebook only, in a separate section, where you can add the answers in markdown cells.
- **For Google Collab Users:**

        - To submit your Colab work, download it as a PDF by navigating to File > Print > Save as PDF before uploading.
        - **Please note:** Submitting a PDF containing a Colablink is not accepted and your submission will not be accepted
- **Single Submission Policy:** Only one submission is allowed per user.